

COMPLETE SETUP GUIDE (Start to Finish)

Step 1: Install Ansible (Control Node)

```
○ ○ ○  
  
1 # Ubuntu/Debian  
2 sudo apt update && sudo apt install ansible sshpass -y  
3  
4 # RHEL/CentOS/Rocky  
5 sudo yum install epel-release -y  
6 sudo yum install ansible sshpass -y  
7  
8 # macOS  
9 brew install ansible  
10  
11 # Windows (WSL2)  
12 sudo apt update && sudo apt install ansible sshpass -y  
13  
14 # Verify Installation  
15 ansible --version
```

Step 2: SSH Key Setup (One-Time)

```
○ ○ ○  
  
1 # Generate SSH Key (if you don't have one)  
2 ssh-keygen -t rsa -b 4096 -f ~/.ssh/ansible_key -N ""  
3  
4 # Copy to ONE test server first  
5 ssh-copy-id -i ~/.ssh/ansible_key.pub user@your-server-ip  
6  
7 # Test Connection  
8 ssh -i ~/.ssh/ansible_key user@your-server-ip
```

Step 3: Project Structure

```
○ ○ ○  
  
1 # Create project folder  
2 mkdir ansible-remote-scripts && cd ansible-remote-scripts  
3  
4 # Create all necessary files  
5 touch ansible.cfg inventory.ini remote-script.yml deploy.sh
```

COMPLETE CODE FILES

1. ansible.cfg (Configuration)

```
○ ○ ○ ansible.cfg  
  
1 [defaults]  
2 inventory = ./inventory.ini  
3 host_key_checking = False  
4 gathering = smart  
5 ansible_python_interpreter = /usr/bin/python3  
6 private_key_file = ~/.ssh/ansible_key  
7  
8 [privilege_escalation]  
9 become = True  
10 become_method = sudo  
11 become_user = root  
12 become_ask_pass = False
```

2. `inventory.ini` (Your Server List)

```
○ ○ ○ inventory.ini

1 # Single server for testing
2 [test]
3 192.168.1.100
4
5 # Multiple servers
6 [webservers]
7 web1.example.com
8 web2.example.com
9 web3.example.com
10
11 [dbservers]
12 db1.example.com
13 db2.example.com
14
15 # Group variables
16 [webservers:vars]
17 ansible_user=ubuntu
18 ansible_ssh_private_key_file=~/.ssh/ansible_key
19
20 [all:vars]
21 ansible_python_interpreter=/usr/bin/python3
```

3. `deploy.sh` (Your Shell Script)

```
○ ○ ○ deploy.sh

1  #!/bin/bash
2  # Sample Deployment Script
3  echo "=====
4  echo " Deployment started on: $(hostname)"
5  echo " Timestamp: $(date)"
6  echo "=====
7
8  # System Information
9  echo "System Overview:"
10 echo "- CPU Cores: $(nproc)"
11 echo "- Memory: $(free -h | awk '/^Mem:/{print $2}')"
12 echo "- Disk: $(df -h / | awk 'NR==2{print $4}') free"
13
14 # Application Deployment Logic
15 echo "Installing dependencies..."
16 # Add your actual commands here
17 # sudo apt update
18 # sudo apt install -y nginx
19
20 echo "Deployment completed successfully!"
21 echo "Status: $(systemctl is-active nginx 2>/dev/null || echo 'Service check skipped')"
22 echo "=====
```

4. remote-script.yml (MAIN PLAYBOOK)

```
remote-script.yml

1 ---
2 - name: Execute Script on Remote Servers
3   hosts: all # Runs on ALL servers in inventory
4   become: yes # Run with sudo privileges
5
6   tasks:
7     # Upload script to remote server
8     - name: Copy script to remote server
9       ansible.builtin.copy:
10         src: deploy.sh
11         dest: /tmp/deploy.sh
12         owner: root
13         group: root
14         mode: '0755' # Make executable
15         register: copy_result
16
17     # Execute the script
18     - name: Run deployment script
19       ansible.builtin.shell:
20         cmd: /tmp/deploy.sh
21         register: script_output
22         args:
23           chdir: /tmp # Change directory before running
24           async: 45 # Timeout in seconds
25           poll: 5 # Check every 5 seconds
26
27     # Display beautiful output
28     - name: Show execution results
29       ansible.builtin.debug:
30         msg: |
31           =====
32           Script executed on: {{ inventory_hostname }}
33           Exit Code: {{ script_output.rc }}
34           Duration: {{ script_output.delta }}
35           =====
36           {{ script_output.stdout }}
37           =====
38
39     # Handle failures gracefully
40     - name: Notify on failure
41       ansible.builtin.debug:
42         msg: " Script failed on {{ inventory_hostname }}"
43       when: script_output.rc != 0
44
45     # Cleanup (optional)
46     - name: Remove script after execution
47       ansible.builtin.file:
48         path: /tmp/deploy.sh
49         state: absent
50       when: script_output.rc == 0
```

EXECUTION COMMANDS

Basic Execution

```
remote-script.yml

1 # Test connection first
2 ansible all -m ping
3
4 # Run on ALL servers
5 ansible-playbook remote-script.yml
6
7 # Run only on webserver group
8 ansible-playbook remote-script.yml --limit webserver
9
10 # Run on specific server
11 ansible-playbook remote-script.yml --limit 192.168.1.100
```

Dry Run (Test Only)

```
remote-script.yml

1 # See what would happen without making changes
2 ansible-playbook remote-script.yml --check --diff
```

Advanced Usage

```
remote-script.yml

1 # Run with verbose output (debugging)
2 ansible-playbook remote-script.yml -vvv
3
4 # Run with tags
5 ansible-playbook remote-script.yml --tags "deploy,cleanup"
6
7 # Parallel execution (10 servers at once)
8 ansible-playbook remote-script.yml -f 10
9
10 # Save output to file
11 ansible-playbook remote-script.yml | tee deployment.log
```

How You Can Use on Different Use Cases

Daily Automation Tasks

```
remote-script.yml

1 # backup.yml - Backup databases on all servers
2 - name: Backup MySQL databases
3   shell: mysqldump -u root --all-databases > /backups/db-$(date +%Y%m%d).sql
4   become: yes
5
6 # update.yml - Security updates
7 - name: Update all servers
8   apt:
9     update_cache: yes
10    upgrade: dist
11    become: yes
12
13 # monitor.yml - Health checks
14 - name: Check disk space
15   shell: df -h | grep -E '^(Filesystem|/[sv]da)'
16   register: disk_space
```

Production Deployment Pipeline

```
remote-script.yml

1 # 1. Deploy code
2 - command: git pull origin main
3   args:
4     chdir: /var/www/app
5
6 # 2. Install dependencies
7 - pip:
8     requirements: /var/www/app/requirements.txt
9
10 # 3. Restart services
11 - systemd:
12     name: gunicorn
13     state: restarted
14     daemon_reload: yes
```

TROUBLESHOOTING GUIDE

Error	Solution
"Permission denied"	<code>chmod 600 ~/.ssh/ansible_key</code>
Python not found	Install Python3 on remote: <code>sudo apt install python3-minimal</code>
SSH timeout	Check firewall: <code>sudo ufw allow 22</code>
Sudo password needed	Add user to sudoers without password
Host unreachable	Verify IP in inventory and network connectivity

BENEFITS & METRICS

Before Ansible	After Ansible	Improvement
10 servers × 5 min each	1 command × 30 seconds	10x faster
Manual errors common	Consistent every time	100% accuracy
No audit trail	Full execution logs	Complete visibility
One-by-one updates	Parallel execution	Scalable to 1000+ servers